

Apuntes de
Diseño y Análisis de Algoritmos
Tomados como estudiante y revisados como
profesor

Federico Melo Barrero
f.melo@uniandes.edu.co

Copyright © 2022, 2026 Federico Melo Barrero.

Todos los derechos reservados.

Ninguna parte de este documento, incluyendo el diseño de la portada, figuras e imágenes, puede ser reproducida, registrada, almacenada o transmitida en manera alguna ni por ningún medio, ya sea electrónico, eléctrico, fotoquímico, mecánico, electroóptico, magnético, de grabación, de fotocopia o cualquier otro sin permiso previo y por escrito del autor.

Índice

I	Introducción	5
1.	Algoritmo	5
2.	Seudocódigo	5
3.	Problema	6
3.1.	Tipos de problemas	6
3.1.1.	Problemas de optimización	7
II	Diseño de algoritmos	8
4.	Dividir y conquistar	8
4.1.	Merge sort	8
4.1.1.	Dividir y conquistar el arreglo	8
4.1.2.	Combinar las mitades	9
4.2.	Subarreglo máximo	10
4.2.1.	Solución por fuerza bruta	11
4.2.2.	Solución por dividir y conquistar	11
5.	Programación dinámica	13
5.1.	Sucesión de Fibonacci	14
5.1.1.	Algoritmo recursivo	14
5.1.2.	Programación dinámica top-down	14
5.1.3.	Programación dinámica bottom-up	15
III	Análisis de algoritmos	16
6.	Correctitud	16
7.	Eficiencia	16
8.	Intratabilidad	17
8.1.	P vs NP	18
8.2.	Problemas de decisión	18

Prefacio

Tomé estos apuntes cuando cursé Diseño y Análisis de Algoritmos en la Universidad de los Andes, entre agosto y noviembre de 2022.

Posteriormente los fui corrigiendo y extendiendo intermitentemente a medida que trabajaba en temas relacionados con algoritmia. Las extensiones más notables corresponden a dos momentos: cuando cursé Teoría de Grafos entre agosto y noviembre de 2023 y cuando fui seleccionado para dictar el curso de Diseño de Algoritmos en mayo de 2026.

Los apuntes están presentados en el mismo formato en el que originalmente fueron escritos: la clase de $\text{\LaTeX}$ que establece su estilo la escribí en noviembre del 2020. Más importante aún, los apuntes preservan la estructura y el tono de cuando los escribí como estudiante, con la intención de que sean útiles precisamente a esa audiencia.

La selección deliberada de la audiencia es en gran medida lo que otorga valor a estas notas. Textos clásicos como CLRS¹ intentan dirigirse a una audiencia mixta: estudiantes, profesores, investigadores y profesionales. Esa amplitud de audiencia genera que el texto se difunda ampliamente, pero también implica un nivel de formalidad, profundidad y exhaustividad que muchas veces no es el más adecuado para un primer encuentro con el tema, especialmente a nivel de pregrado.

Estas notas no sacrifican rigor ni correctitud, pero sí priorizan claridad pedagógica sobre exhaustividad formal. En consecuencia, omiten algunos detalles cuya inclusión sería valiosa en un texto de referencia, pero que para muchos estudiantes solo alargarían el texto y añadirían complejidad innecesaria.

¹Uso esta sigla para referirme al famosísimo Introduction to Algorithms de Cormen, Leiserson, Rivest y Stein, que es un excelente ejemplo de este fenómeno: con docenas de miles de citas en google scholar, claramente tiene la profundidad suficiente para resultar útil a investigadores.

Prerrequisitos

Estos apuntes están escritos de forma que los prerrequisitos para entenderlos son sorprendentemente laxos para el nivel de material que se cubre. Hay que saber lo siguiente:

- Programación básica en algún lenguaje de programación, incluyendo recursión.
 - El pseudocódigo que se emplea es bastante parecido a Python en la sintaxis; sin embargo, es legible si se tiene experiencia con cualquier lenguaje de programación imperativo.
- Algo de teoría sobre las estructuras de datos más básicas: arreglos, listas, etc. En qué difieren y cuáles son las ventajas de una sobre otra.
- Matemática básica
 - Notación matemática básica: conjuntos, operadores, cuantificadores, etc.
 - Aritmética básica: sumar, multiplicar, potenciar, etc.
 - Álgebra elemental: despejar variables, factorizar, etc.
 - Funciones elementales: logaritmos, exponenciales, polinomios, etc.

Parte I

Introducción

1. Algoritmo

Estas notas estudian algoritmos. Un **algoritmo** es una secuencia de pasos. Pero nos interesan los algoritmos que resuelven un problema de computación, entonces:

Definición 1.1. Algoritmo

Secuencia de pasos para transformar un conjunto de valores (**entrada/input**) a un conjunto de valores (**salida/output**).

No se escribe “a otro conjunto de valores” porque la salida podría ser igual a la entrada.

Un algoritmo se puede expresar en:

- Lenguaje natural (español, inglés, etc.)
- Lenguaje formal (matemática o algún lenguaje de programación o lógico)
- Seudocódigo (una mezcla de los dos anteriores)
- Como programa de computador
- Como hardware (esto es difícil de imaginar en este punto, pero sí)

2. Seudocódigo

Elseudocódigo es básicamente algo que se parece a un lenguaje de programación pero que permite:

- Despreocuparse de detalles de sintaxis o de implementación, como el manejo de errores.
- Usar la forma más clara de expresarse en cada línea: a veces español, a veces matemática, a veces sintaxis de algún lenguaje de programación.

Entonces es muy útil para expresar algoritmos.

En los textos formales se definen convenciones estrictas para elseudocódigo que se usa; eso elimina la ambigüedad, pero obliga al lector a aprenderse las convenciones, prácticamente un nuevo lenguaje. Contrariamente, en estos apuntes se utiliza unseudocódigo menos estructurado, basado en la sintaxis de Python, pero con varias libertades. Las ambigüedades se eliminan aclarándolas cuando aparecen.

En Python son muy idiomáticas las instrucciones de alto nivel, que automáticamente ordenan, buscan, etc. y que no existen en otros lenguajes. Ese tipo de instrucciones se omiten deliberadamente, primero porque formalmente arruinarían el análisis algorítmico, como se expone más adelante, y segundo porque la idea es precisamente entender cómo implementar eficientemente ese tipo de instrucciones. Así que, bajo esa perspectiva, elseudocódigo es más parecido a un lenguaje de programación de mediano o bajo nivel.

E.g., una función que recibe un arreglo de números y encuentra el mayor:

```

1 función encontrar_mayor(números: arreglo de enteros) entero
2     mayor = -∞
3     for i=0 to números.longitud-1
4         if números[i] > mayor
5             mayor = números[i]
6     return mayor

```

Detalles de este pseudocódigo que reflejan que su propósito es ser claro:

- Algunas palabras y tipos están en español. A pesar de que no se definieron de antemano, son muy claros porque son los términos que usamos para referirnos a esas cosas.
- Omite detalles de sintaxis como los dos puntos, pero preservo los que otorgan claridad, como la indentación.
- Evito sintaxis específica de algún lenguaje.
 - En el for uso una sintaxis que hace explícito el valor inicial y el valor final sin necesidad de usar iterables (y sin tener que desambiguar si se toma o no el último valor; acá es claro que sí)
 - Presumo la existencia de un método `longitud` para obtener la longitud de un arreglo. Es una presunción razonable porque en cualquier lenguaje orientado a objetos existe algo así.

3. Problema

Un problema (en este contexto) es un enunciado que especifica qué se quiere obtener, sin decir cómo. Más formalmente:

Definición 3.1. Problema (computacional)

Especificación de una entrada y de una salida con base en la entrada.

E.g., el **problema de ordenamiento** (*sorting problem*, muy famoso):

- **Entrada:** secuencia de n números $a_1, a_2, \dots, a_n$.
- **Salida:** permutación $a'_1, a'_2, \dots, a'_n$ tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$

Una **permutación** de una secuencia es un posible orden de sus elementos; es como decir, coloquialmente, un orden. Si la secuencia tiene más de un elemento, tiene muchas permutaciones. En este caso, el problema especifica como salida una permutación en la que los números están ordenados de menor a mayor.

Una **instancia** de un problema es una entrada específica, e.g., 1, 4, 2, 3, 2 es una instancia del problema de ordenamiento, en cuyo caso una **solución** es 1, 2, 2, 3, 4.

3.1. Tipos de problemas

Hay muchos tipos de problemas computacionales. En general, un mismo problema de la vida real se puede modelar de varias formas, que pueden corresponder a un tipo de problema computacional diferente. Algunos tipos son:

- **Problemas de enumeración:** la salida son todas las soluciones válidas.
- **Problemas de conteo:** la salida es el número de soluciones válidas.
- **Problemas de optimización:** la salida es la mejor solución válida de acuerdo a algún criterio.
- **Problemas de búsqueda:** la salida es alguna solución válida para el problema, si existe.
- **Problemas de decisión:** la salida es un sí o un no.

Estas categorías no son mutuamente excluyentes, muchas veces son distintas perspectivas del mismo problema. Por ejemplo, para el problema de ordenamiento se puede abordar bajo cualquiera de las perspectivas anteriores:

- Enumeración: dado un arreglo, listar todas las permutaciones ordenadas de sus elementos.
- Conteo: dado un arreglo, determinar cuántas permutaciones distintas de sus elementos están ordenadas.

- Optimización: dado un arreglo y un conjunto de operaciones permitidas, encontrar una secuencia de operaciones que lo ordene usando el menor número de operaciones posible.
- Búsqueda: dado un arreglo, encontrar alguna permutación de sus elementos que esté ordenada.
- Decisión: dado un arreglo, ¿está ordenado?

Se pueden formular aún más problemas relacionados a ordenar elementos que encajen en cualquiera de estas categorías. En este punto, no es tan inmediato determinar cómo traducir entre tipos de problemas, en gran parte porque las definiciones anteriores no son matemáticamente formales, sino más bien la intuición detrás de cada tipo de problema. Por lo pronto, al diseñar algoritmos nos ocuparemos de indicar qué tipo de problemas se pueden abordar con la técnica que se está presentando, sin perder de vista que muchos problemas admiten reformulaciones que los hacen encajar en distintos tipos.

3.1.1. Problemas de optimización

Antes de iniciar

Parte II

Diseño de algoritmos

En esta parte de las notas se presentan distintas técnicas para diseñar algoritmos. No se presentan en orden de dificultad, principalmente debido a que la dificultad es subjetiva: cada método es difícil por una razón distinta. Pero sí se presentan en un orden conveniente, para que al explicar métodos posteriores se puedan dar por sentados conceptos de métodos anteriores.

4. Dividir y conquistar

Muchos problemas pueden dividirse en subproblemas más pequeños y esos subproblemas siguen siendo muy similares al problema original. Por ejemplo, organizar su casa se puede dividir en organizar la cocina, la sala, etc. que son problemas parecidos pero con menos cosas para organizar.

Dividir y conquistar es la técnica de diseño de algoritmos que formaliza esa idea. Consiste en dividir el problema en subproblemas, resolver cada uno de ellos y luego combinar las soluciones para obtener la solución del problema original. Formalmente, las etapas son:

- **Dividir:** Dividir el problema en subproblemas más pequeños
- **Conquistar:** Resolver cada subproblema de forma recursiva (o, si el subproblema es muy pequeño, resolverlo directamente)
- **Combinar:** Combinar las soluciones de los subproblemas, obteniendo la solución del problema original

Se presentan tres algoritmos para concretar esta idea.

4.1. Merge sort

Merge sort resuelve el problema de ordenamiento (definido en la sección 3).

La idea es dividir el arreglo a ordenar en dos mitades, ordenar cada mitad de forma recursiva (es decir, dividiendo cada una otra vez a la mitad, y así sucesivamente, hasta que queden pedazos de un elemento, que ya están ordenados) y luego combinar las soluciones.

Haciendo explícitas las etapas:

- **Dividir:** Dividir el arreglo en dos mitades.
- **Conquistar:** Ordenar cada mitad usando merge sort (o sea, aplicando estas etapas a cada mitad)
- **Combinar:** Combinar las soluciones de cada mitad, que son subarreglos ordenados.

4.1.1. Dividir y conquistar el arreglo

Dividir el arreglo no tiene ciencia: se calcula la mitad con división entera y con eso se conocen los índices de cada segmento. La etapa de conquistar es simplemente hacer el llamado recursivo a cada mitad, porque en algún punto al dividir continuamente por la mitad se obtienen arreglos de longitud uno, que ya están ordenados.

Por ahora, para no pensar en cómo combinar las soluciones, encapsulamos ese paso en una función que se implementará después. Con eso, este es el pseudocódigo:


```

1 función merge_sort(números: arreglo, ind_min: entero, ind_max: entero)
2   if ind_min >= ind_max      { No se retorna nada porque el algoritmo es in-place; se usa el
3     return                  { return para detener la función en el caso base de la recursión
4
5   ind_mitad = (ind_min + ind_max) // 2
6   merge_sort(números, ind_min, ind_mitad)
7   merge_sort(números, ind_mitad+1, ind_max)
8
9   combinar(números, ind_min, ind_mitad, ind_max)

```

Necesitamos tener los índices como parámetros porque indican qué segmento del arreglo se está procesando. La invocación inicial, que resuelve el problema completo, usa los índices extremos del arreglo:

```
merge_sort(números, 0, números.longitud-1)
```

Nótese que el arreglo no se divide físicamente en cada llamado recursivo; únicamente se delimitan distintas regiones mediante índices. Esto es para que el algoritmo sea **in-place**, es decir, modifique directamente el arreglo original, sin crear y retornar uno nuevo. En cada llamado recursivo ocurre lo siguiente:

1. Si el segmento tiene longitud 1 (o 0), se alcanza el caso base y la función termina.
2. Si no, el segmento se divide conceptualmente en dos mitades y se realizan llamados recursivos para la mitad izquierda y luego la derecha. El llamado a la mitad derecha empieza después de que termina el llamado a la mitad izquierda.
3. Una vez ambas llamadas retornan, se ejecuta `combinar`, que mezcla ambas mitades ordenadas in-place dentro del rango delimitado por `ind_min` e `ind_max`.

Para arreglos largos, el caso base solo se alcanza después de muchos llamados recursivos, cuando el arreglo se ha dividido lo suficiente. Es decir: en las primeras invocaciones, siempre ocurre que el segmento se divide conceptualmente. Se acumulan llamados a `merge_sort` uno encima de otro, sin retornar ni `combinar`. Ese es el **descenso** de la recursión.

Cuando algún llamado alcanza el caso base, ocurre el primer retorno. La ejecución continúa justo después de ese llamado recursivo (o sea, al final de la línea 5 o de la línea 6) y entonces pasa una de dos cosas: o se realiza otro llamado recursivo (que pronto alcanzará el caso base, porque el segmento ya es muy pequeño), o se ejecuta por primera vez el paso de `combinar`, mezclando esos dos segmentos (que son pequeños, porque hace poco se llegó al caso base). Después de `combinar`, la función termina y retorna a quien la había invocado. Así, los llamados a `merge_sort` que se habían acumulado empiezan a retornar uno tras otro y la recursión comienza a **desenrollarse**. Ese es el **ascenso** de la recursión.

4.1.2. Combinar las mitades

Ahora, solo falta pensar en cómo combinar las mitades. Sorprendentemente, la mejor forma de combinarlas es la forma más intuitiva. Si cada arreglo fuera una pila de cartas, el método natural sería comparar la primera de cada pila y tomar la menor; luego comparar la siguiente de esa pila con la primera de la otra y tomar la menor (que en la combinación es la que le sigue a la primera que se había tomado), y así sucesivamente. El pseudocódigo es el siguiente:

```

1 función combinar(números: arreglo, ind_min: entero, ind_mitad: entero, ind_max: entero)
2   izquierda = copia de números desde ind_min hasta ind_mitad
3   derecha = copia de números desde ind_mitad+1 hasta ind_max
4   i = 0
5   j = 0
6
7   while i < izquierda.longitud and j < derecha.longitud
8     if izquierda[i] <= derecha[j]
9       números[ind_min+i+j] = izquierda[i]
10      i++
11     else
12       números[ind_min+i+j] = derecha[j]
13       j++
14
15   while i < izquierda.longitud
16     números[ind_min+i+j] = izquierda[i]
17     i++
18
19   while j < derecha.longitud
20     números[ind_min+i+j] = derecha[j]
21     j++

```

Se crean arreglos nuevos con cada mitad. La complejidad espacial es entonces $\Theta(n)$.

<= en lugar de < para estabilidad.

Lógica principal, cuando ambos subarreglos tienen elementos: se toma el menor y se añade a números.

Si quedan elementos en el arreglo de la izquierda, ya no quedan en el de la derecha. Se añade el excedente de la izquierda a números.

Se añade el excedente de la derecha a números. Si este while corre, es porque el anterior no corrió y viceversa.

Un algoritmo es **estable** si mantiene el orden relativo de los elementos iguales. O sea, en caso de empate, deja primero al que estaba primero en el arreglo original. En la línea del comentario sobre estabilidad, se elige el elemento de la izquierda en caso de empate para que continúe ubicado antes que el de la derecha en la solución final.

Práctica: Sort an Array

4.2. Subarreglo máximo

Dado un arreglo de números, a un subarreglo contiguo no vacío cuya suma (de sus elementos) es mayor que la de cualquier otro subarreglo contiguo se le denomina un **subarreglo máximo**. El problema de encontrar un subarreglo máximo se define como:

- **Entrada:** arreglo con n números: $[a_1, a_2, \dots, a_n]$.
- **Salida:** subarreglo contiguo $[a_i, a_{i+1}, \dots, a_j]$ no vacío tal que la suma $\sum_{k=i}^j a_k$ es máxima.

“es máxima” se puede formalizar en una expresión matemática que básicamente dice que esa sumatoria es mayor que cualquier otra sumatoria de elementos contiguos que se pueda formular. Nótese que podría haber más de un subarreglo máximo: podrían haber varios cuya suma sea igual. Basta con hallar cualquiera que sea máximo.

Hay dos tipos de instancias de este problema que son muy fáciles:

- Cuando todos los valores del arreglo son positivos, la respuesta es todo el arreglo, porque todos los valores aportan a una mayor suma.
- Cuando todos los valores son negativos, se quiere que la suma sea lo menos negativa, por lo que se forma el subarreglo con un solo elemento, el menos negativo.

Para facilitar la implementación, se transforman levemente las especificaciones del problema por unas equivalentes:

- **Entrada:** arreglo con n números: $[a_0, a_2, \dots, a_{n-1}]$.
- **Salida:** índices i, j del arreglo, donde $0 \leq i \leq j < n$ tales que la suma $\sum_{k=i}^j a_k$ es máxima.

Los cambios son dos: primero, se utilizan índices basados en 0 para el arreglo (menos usual en notación matemática, pero más compatible con lenguajes de programación) y segundo, ahora se buscan los índices que delimitan el máximo subarreglo, en lugar del subarreglo como tal.

4.2.1. Solución por fuerza bruta

Una primera solución a este problema se consigue simplemente comparando todas las posibilidades de subarreglos y eligiendo el que tiene la mayor suma:

```

1 función subarreglo_max_fuerza_bruta(numeros: arreglo) tripla de enteros
2     mayor = -∞
3     ind_ini = 0
4     ind_fin = 0
5
6     for i=0 to numeros.longitud-1
7         suma = 0
8         for j=i to numeros.longitud-1
9             suma += numeros[j]
10
11             if suma > mayor
12                 mayor = suma
13                 ind_ini = i
14                 ind_fin = j
15
16     return ind_ini, ind_fin, mayor

```

Ese algoritmo cuenta con dos ciclos anidados que recorren el arreglo, por lo que intuitivamente su complejidad temporal es $O(n^2)$.

4.2.2. Solución por dividir y conquistar

Para diseñar un algoritmo usando dividir y conquistar, el primer paso convencional es partir el arreglo. Con eso, cada subproblema es hallar un subarreglo máximo en el respectivo pedazo. El cuidado que hay que tener con esa solución es que, al partir el arreglo, podríamos tener el infortunio de partirlo justo donde estaba el subarreglo máximo, entonces en el paso de combinar las soluciones, tenemos que asegurarnos de contemplar esa posibilidad. Con eso, las etapas son:

- **Dividir:** Dividir el arreglo en dos mitades.
- **Conquistar:** Para cada mitad, hallar un subarreglo máximo recursivamente.
- **Combinar:** Comparar los subarreglos máximos de cada mitad para elegir el mayor y también compararlos con los subarreglos que pasan por donde se dividió el arreglo.

Si encapsulamos la última parte del paso de combinar en `subarr_max_transversal`, de forma que aplazamos un poco ese problema, podemos escribir el pseudocódigo para las primeras dos etapas:

```

1 función subarr_max_div_conquistar(
2     numeros: arreglo,
3     ind_min: entero,
4     ind_max: entero
5 ) tripla de enteros
6     if ind_min == ind_max
7         return ind_min, ind_max, numeros[ind_min]
8
9     ind_mitad = (ind_min + ind_max) // 2
10
11     ini_izq, fin_izq, suma_izq = subarr_max_div_conquistar(numeros, ind_min, ind_mitad)
12     ini_der, fin_der, suma_der = subarr_max_div_conquistar(numeros, ind_mitad+1, ind_max)
13     ini_mitad, fin_mitad, suma_mitad = subarr_max_transversal(numeros, ind_min, ind_mitad, ind_max)
14
15     if suma_izq >= suma_der and suma_izq >= suma_mitad
16         return ini_izq, fin_izq, suma_izq
17     if suma_der > suma_mitad
18         return ini_der, fin_der, suma_der
19     return ini_mitad, fin_mitad, suma_mitad

```

Ahora sí, pensemos los subarreglos que pasan por donde se dividió el arreglo. Todos esos subarreglos tienen un pedazo a la izquierda de esa mitad y un pedazo a la derecha. Para maximizar la suma, queremos maximizar la de la derecha y maximizar la de la izquierda. En este caso, usar un algoritmo exhaustivo no es ingenio:

```

1 función subarr_max_transversal(
2     numeros: arreglo,
3     ind_min: entero,
4     ind_mitad: entero,
5     ind_max: entero
6 ) tripla de enteros
7     suma_der, suma_izq = 0, 0
8     mayor_der, mayor_izq = -∞, -∞
9
10     for i=ind_mitad to ind_max
11         suma_der += numeros[i]
12         if suma_der > mayor_der
13             mayor_der = suma_der
14             extremo_der = i
15
16     for j=ind_mitad-1 downto ind_min
17         suma_izq += numeros[j]
18         if suma_izq > mayor_izq
19             mayor_izq = suma_izq
20             extremo_izq = j
21
22     return extremo_izq, extremo_der, mayor_izq+mayor_der

```

Práctica: Maximum Subarray

Ejercicios

Calentamiento

- Longest Common Prefix

Estándar

- Kth Largest Element in an Array
- Search a 2D Matrix II

Duros

- Median of Two Sorted Arrays
- The Skyline Problem
- Reverse Pairs

5. Programación dinámica

La **programación dinámica**² es una técnica avanzada de diseño de algoritmos.

Se puede ver como una mejora de dividir y conquistar para ciertos problemas. En ocasiones, al usar dividir y conquistar, los subproblemas en los que se dividió el problema original comparten subsubproblemas idénticos. Al resolver cada subproblema por aparte, los subsubproblemas iguales se resuelven varias veces. La idea fundamental de la programación dinámica es solo resolver cada subsubproblema una vez y almacenar su resultado en una estructura de datos para poder reutilizarlo.

En ese sentido, se realiza un *trade-off* de complejidad espacial por complejidad temporal: se gasta más espacio para un algoritmo más veloz.

Se puede aplicar programación dinámica a muchos problemas muy distintos, pero no a todos se puede y no a todos tiene sentido hacerlo. Primeramente, usualmente se aplica a problemas planteados como **problemas de optimización**: problemas que tienen un conjunto de soluciones factibles, cada una con un valor, y en los que se busca una **solución óptima**: una cuyo valor sea mínimo o máximo, acorde. Sumado a eso, los problemas que se resuelven eficientemente con programación dinámica tienen dos propiedades fundamentales:

1. Los problemas *deben* tener **subestructura óptima**: que la solución óptima del problema original se pueda construir a partir de soluciones óptimas a sus subproblemas.
2. Para que la programación dinámica mejore la eficiencia, el problema debe presentar **subproblemas superpuestos**, es decir, que un algoritmo recursivo para el problema acabaría resolviendo los mismos subproblemas varias veces.
 - Para que eso ocurra, el espacio de posibles subproblemas debe ser pequeño, usualmente polinomial en el tamaño de la entrada.
 - Si el problema no presenta subproblemas superpuestos, usar programación dinámica no es más eficiente que usar dividir y conquistar.

Hay en esencia dos formas de diseñar algoritmos de programación dinámica:

- **Top-down** (del problema más general a los más pequeños): Se resuelve el problema original de forma recursiva, pero cada vez que se resuelve un subproblema, se almacena su resultado para evitar tener que recomputarlo. A eso se le llama **memoización**³.
 - Se crea una estructura de datos para almacenar los resultados de cada llamado recursivo. Se llena de elementos iniciales que indiquen que el resultado en esa posición no se ha computado aún (e.g., null, ∞ , etc.). Para que la información persista entre llamados recursivos, la estructura tiene que estar definida afuera de la función recursiva, si no, cada llamado recursivo crearía su propia estructura, sin los resultados previos. Usualmente, la función recursiva pasa a ser una función auxiliar de una función principal en la que se define la estructura.
- **Bottom-up** (de los problemas pequeños al general): Se resuelven primero los subproblemas más pequeños y se almacenan sus resultados en una tabla. A eso se le denomina **tabulación**. Se usan esos resultados para resolver subproblemas más grandes, hasta llegar al problema original.
 - Como los subproblemas se resuelven en orden, se reemplaza el uso de recursión por un ciclo.

Como ambos métodos aprovechan la propiedad de subproblemas superpuestos, sus complejidades asintóticas suelen ser iguales. Sin embargo, si todos los subproblemas posibles se tienen que resolver al menos una vez, el algoritmo bottom-up suele ser más eficiente que el top-down por un factor constante, porque el algoritmo top-down tiene el sobre costo de la recursión.

Ocasionalmente, no es necesario resolver todos los subproblemas del espacio; en ese caso, la solución bottom-up puede ser peor porque requiere resolver los subproblemas pequeños primero para completar la tabulación,

²El nombre no tiene relación con cómo funciona la técnica; fue un término acuñado por Richard Bellman en la década de 1950 para describir su investigación, deliberadamente ambiguo para evitar la aversión de la Fuerza Aérea estadounidense hacia la investigación matemática en aquel entonces.

³El término "memoización" proviene del inglés *memoization*, que significa convertir los resultados en un memo, abreviación del latín *memorandum*: algo para recordar.

mientras que la solución top-down solo memoiza los subproblemas que sí se tienen que resolver.

A continuación se presentan múltiples ejemplos de problemas que se resuelven eficientemente usando programación dinámica, ordenados en dificultad ascendente.

5.1. Sucesión de Fibonacci

La sucesión de Fibonacci comienza con 0 y 1, y cada número posterior es la suma de los dos anteriores. Formalmente:

$$\begin{aligned} F(0) &= 0 \\ F(1) &= 1 \\ F(n) &= F(n-1) + F(n-2) \quad \text{para } n > 1 \end{aligned}$$

El problema radica en calcular el n -ésimo número de Fibonacci, es decir, $F(n)$, para algún n dado.

5.1.1. Algoritmo recursivo

El algoritmo recursivo, que es el más intuitivo, es una traducción directa de la definición matemática:

```

1 función fibonacci(n: entero) -> entero
2   if n == 0 return 0
3   if n == 1 return 1
4
5   return fibonacci(n-1) + fibonacci(n-2)
```

Su ineficiencia radica en que presenta subproblemas superpuestos: para calcular $F(n)$ se necesitan calcular $F(n-1)$ y $F(n-2)$; para calcular $F(n-1)$ se necesitan calcular $F(n-2)$ y $F(n-3)$. En tan solo dos pasos, ya se ha calculado $F(n-2)$ dos veces. A medida que se avanza, la cantidad de cálculos repetidos crece exponencialmente. De hecho, la complejidad temporal de ese algoritmo es $\Theta(\phi^n)$, donde ϕ es el número áureo, aproximadamente 1.618.

Sabiendo que el algoritmo presenta subproblemas superpuestos, solo falta determinar si presenta subestructura óptima para concluir que se puede resolver eficientemente con programación dinámica. En este caso, sí la presenta: la solución óptima para $F(n)$ se construye a partir de las soluciones óptimas para dos subproblemas, $F(n-1)$ y $F(n-2)$.

5.1.2. Programación dinámica top-down

Una primera optimización es la implementación top-down: incluir memoización en la implementación recursiva para eliminar la redundancia que introducen los problemas superpuestos:

```

1 función fibonacci_top_down(n: entero) entero
2   soluciones = arreglo de enteros de tamaño n + 1
3   soluciones[0] = 0   { Se añaden los casos base
4   soluciones[1] = 1   { directamente al arreglo
5
6   for i=2 to n         { Se llena el resto del arreglo con -1, que indica que no se ha calculado ese término
7     soluciones[i] = -1 { de la secuencia. Es inambiguo porque los términos calculados serán positivos
8
9   return fibonacci_top_down_recursion(n, soluciones) .....> La función auxiliar recursiva recibe siem-
                                                                pre la misma estructura y la va llenando
10
11 función fibonacci_top_down_recursion(n: entero, soluciones: arreglo de enteros) entero
12   if soluciones[n] < 0 .....> La estructura aún no contiene la solución para este valor, entonces, se memoiza
13     soluciones[n] = fibonacci_top_down(n-1, soluciones) + fibonacci_top_down(n-2, soluciones)
14
15   return soluciones[n] .....> Se retorna siempre el valor memoizado (que puede que se
                                                                haya calculado en ese llamado recursivo o en uno anterior)
```

Por definición, para calcular el n -ésimo número de Fibonacci es necesario calcular todos los anteriores, por ende, el algoritmo top-down es menos eficiente que el algoritmo bottom-up: ambos calculan todos los términos anteriores, pero el bottom-up evita el uso de recursión.

5.1.3. Programación dinámica bottom-up

El método bottom-up resuelve primero los problemas más pequeños. En este caso, eso traduce naturalmente a calcular primero los menores términos de la sucesión, o lo que es lo mismo, a calcularlos en orden:

```
1 función fibonacci(n: entero) entero
2     soluciones = arreglo de enteros de tamaño  $n + 1$ 
3     soluciones[0] = 0
4     soluciones[1] = 1
5
6     for i=2 to n:
7         soluciones[i] = soluciones[i-1] + soluciones[i-2]
8
9     return soluciones[n]
```

Como optimización adicional, este problema tiene la particularidad de que no hace falta preservar las soluciones previas, de forma que se puede ahorrar el espacio en memoria que ocupa el arreglo de soluciones:

```
1 función fibonacci(n: entero) entero
2     solucion_trasanterior = 0
3     solucion_anterior = 1
4
5     for i=2 to n:
6         solucion = solucion_anterior + solucion_trasanterior
7         solucion_trasanterior = solucion_anterior
8         solucion_anterior = solucion
9
10    return solucion
```

Se obtiene un algoritmo cuya complejidad temporal es $\Theta(n)$ y cuya complejidad espacial es $\Theta(1)$.

Parte III

Análisis de algoritmos

Analizar un algoritmo consiste en determinar si es correcto y predecir qué recursos consumiría. Usualmente, el recurso más relevante es el tiempo de ejecución, pero a veces vale la pena analizar el espacio en memoria o algún otro recurso.

El análisis es importante porque uno quiere que su algoritmo sea correcto y rápido. Eso suele ser difícil; a menudo hay algoritmos correctos pero lentos o algoritmos rápidos pero probabilísticos.

La sección ?? se ocupa de la correctitud, mientras que la sección ?? se ocupa del tiempo de ejecución (y expone conceptos que se pueden usar para analizar otros recursos, de ser necesario).

6. Correctitud

Definición 6.1. Correctitud

Un algoritmo es **correcto** si para toda instancia de entrada arroja la salida correcta, es decir, la salida que satisface la relación con la entrada definida por el problema.

Como la correctitud se debe verificar para todas las posibles instancias, usualmente exige una demostración matemática.

7. Eficiencia

Entre dos algoritmos correctos que resuelven un mismo problema, naturalmente se prefiere el que lo resuelve más rápidamente.

Dado un algoritmo, (usualmente) es posible realizar una **implementación**: un programa que ejecuta el algoritmo. El tiempo de ejecución de una implementación de un algoritmo depende de:

1. El lenguaje de programación en el que se implementa.
2. La máquina en la que se ejecuta.
3. Los datos que recibe el programa como entrada.
4. El número de operaciones que realiza el algoritmo.

Al analizar un algoritmo, tiene sentido ignorar 1 y 2, pues la máquina y el lenguaje pueden variar. Entonces, lo que se busca es conocer el número de operaciones que hace el algoritmo, que usualmente depende del tamaño de los datos que recibe como entrada. Así pues, el objetivo del análisis de **eficiencia** es obtener una expresión matemática para la cantidad de operaciones que hace un algoritmo en función del tamaño de su entrada.

1. Asignar un tiempo constante a cada instrucción primitiva del algoritmo
2. Contar cuántas veces ocurre cada instrucción
3. Sumar esos tiempos

A eso se le denomina **análisis exacto** de la eficiencia.

8. Intratabilidad

Los problemas que se pueden resolver con un computador se clasifican de acuerdo a la complejidad temporal del mejor algoritmo conocido para resolverlos. Eso da una noción de su dificultad: si alguien ha encontrado un algoritmo polinomial para el problema, se dice que es **tratable**⁴; si nadie lo ha hecho, de forma que solo se conocen algoritmos de tiempo súperpolinomial, se dice que es **intratable**.

Otra forma de comparar problemas es a través de reducciones. Una **reducción** consiste en transformar un problema en otro, tal que resolver ese otro te permita resolver el original.

Definición 8.1. Reducción

Un problema A se puede **reducir** a otro problema B si existe un algoritmo que, para cualquier instancia de A , construya una instancia de B tal que la solución de esa instancia de B permita construir la solución de la instancia original de A .

Si existe una reducción de A a B y esa reducción es polinomial, el algoritmo que resuelve B ahora se puede utilizar para resolver A y entonces se dice que B es igual o más difícil que A . Usualmente, cuando se habla de una reducción, se presume que es polinomial, porque de lo contrario no es útil: si la reducción es más compleja que resolver el problema, no sirve de nada transformarlo.

Con eso, se describe la clasificación formal de problemas, aunque solo más adelante se dan definiciones genuinamente rigurosas para cada clase de problemas. La clasificación es la siguiente:

- **P (polinomial)**: problemas para los que se conoce algún algoritmo polinomial correcto que los resuelva.
- **NP (no determinista polinomial)**: problemas para los que se conoce algún algoritmo polinomial que, dado un candidato de solución, verifique si es correcto.
 - Como todos los problemas que están en P se resuelven polinomialmente, también están en NP , porque también se pueden verificar polinomialmente. Es decir, $P \subseteq NP$.
 - Puede ser que todos los problemas que se pueden verificar en tiempo polinomial también se puedan resolver en tiempo polinomial, es decir, que $P = NP$, pero no se ha demostrado ni refutado esa posibilidad. A esta interrogante se le llama **P vs NP**⁵.
- **NP-Hard (no determinista polinomial difícil)**: problemas tal que cualquier problema en NP se puede reducir en tiempo polinomial a ellos.
 - Como cualquier problema en NP se puede reducir polinomialmente a ellos, se dice que son igual o más difíciles que cualquier problema NP .
 - No necesariamente los problemas NP-Hard son NP ; muchos no se pueden verificar en tiempo polinomial.
 - Si mañana se encuentra un algoritmo polinomial para resolver algún problema NP-Hard, sería revolucionario: cada problema NP se puede reducir polinomialmente a ese problema NP-Hard, que ahora podría solucionarse polinomialmente con ese nuevo algoritmo. Eso implicaría que todo problema NP sería polinomial, o sea que $P = NP$.
- **NP-Completo (no determinista polinomial completo)**: problemas NP-Hard que son NP .
 - Como son NP-Hard, para ninguno se conoce un algoritmo polinomial, pero como son NP , se pueden verificar en tiempo polinomial. En ese sentido, son los problemas más difíciles de NP .
 - Son los más interesantes porque muchos problemas NP Completos son muy similares a problemas que sabemos que son P . Entonces es muy tentador intentar encontrar un algoritmo polinomial

⁴Se usa la palabra tratable como un calco de la palabra inglesa **tractable**, cuya traducción es más cercana a “manejable”

⁵El Clay Mathematics Institute ofrece un premio de un millón de dólares a quien resuelva P vs NP , como parte de sus *Millennium Prize Problems*

para alguno y concluir que $P = NP$.

Resulta evidente que las relaciones de contención entre esas clases dependen de P vs NP . La figura 1 muestra la relación entre las clases de problemas en ambos escenarios.

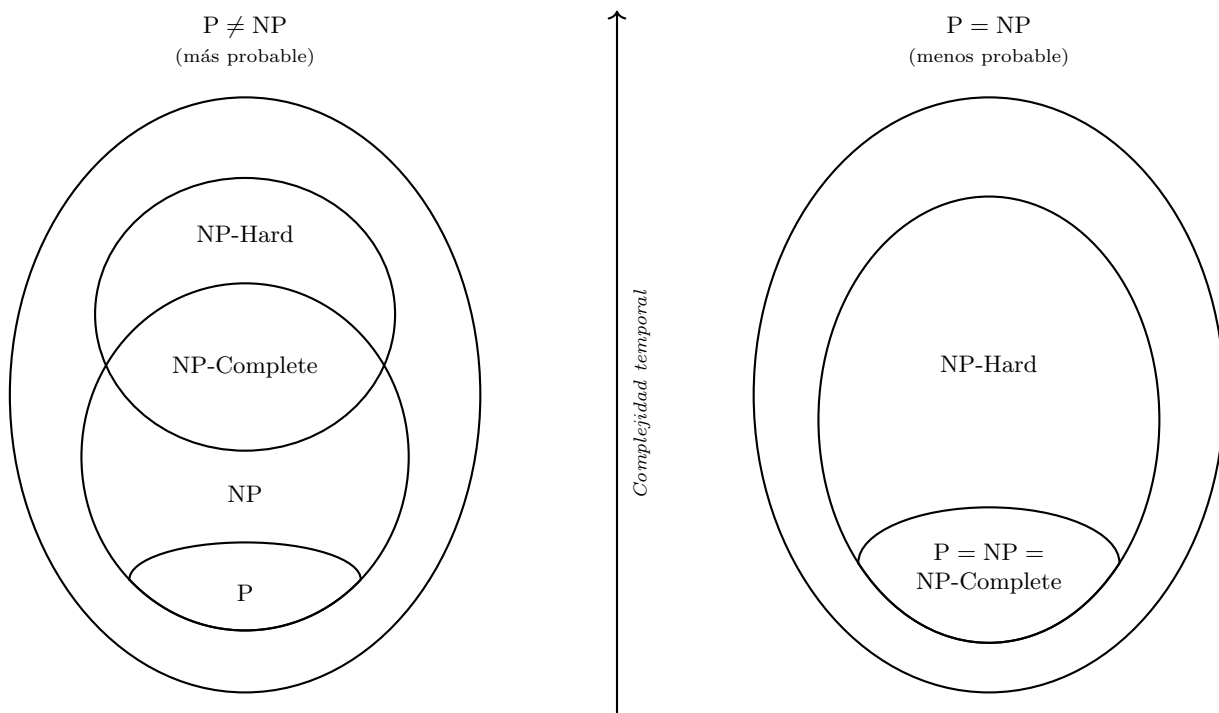


Figura 1: Clasificación de los problemas computables por clases de complejidad, para los dos escenarios del problema abierto P vs. NP .

8.1. P vs NP

Muchos de los problemas NP -Hard y NPC se han estudiado al detalle por décadas, muchas veces individualmente, y nadie ha encontrado un algoritmo polinomial para ninguno. El hecho de que encontrar un algoritmo polinomial para alguno implica que siempre han existido algoritmos polinomiales para todos, implica que sería muy sorprendente que $P = NP$. Sin embargo, hay que contemplar esa posibilidad, por remota que sea.

Para hacer más énfasis en lo anterior, imagine que muchas personas han buscado duendes en su jardín durante años, y nadie ha encontrado ninguno. Eso sugiere que no existen, pero ahora imagine un supuesto más fuerte aún: que si alguien encuentra un duende en su jardín, significa que siempre habrían habido duendes en cada jardín del mundo, pero nadie los había encontrado nunca. Eso significa no solo que quien encontró el duende tuvo gran habilidad o que su jardín era especial, sino también que todos los demás estuvieron rodeados de duendes todo el tiempo sin darse cuenta. Por eso es que se dice que es muy improbable que $P = NP$.

8.2. Problemas de decisión

A pesar de que informalmente se usan las clases P , NP y NP -Hard, para clasificar prácticamente cualquier problema computacional, estas categorías aplican formalmente solo a **problemas de decisión**, es decir, problemas que se pueden responder con un sí o un no.

De esos problemas, no todos se pueden resolver con un computador, incluso cuando se pueden modelar matemáticamente. Los círculos externo de la figura 1 representan únicamente los problemas de decisión que son computables, que se llaman **decidibles**.

Muchos de los problemas de algoritmia son problemas de optimización, en donde